

MTG-GAN: A Masked Temporal Graph Generative Adversarial Network for Cross-Domain System Log Anomaly Detection

Jing Zhang, Chao Luo *member, IEEE*, and Rui Shao

Abstract—Anomaly detection of system logs is crucial for the service management of large-scale information systems. Nowadays, log anomaly detection faces two main challenges: 1) capturing evolving temporal dependencies between log events to adaptively tackle with emerging anomaly patterns, 2) and maintaining high detection capabilities across varies data distributions. Existing methods rely heavily on domain-specific data features, making it challenging to handle the heterogeneity and temporal dynamics of log data. This limitation restricts the deployment of anomaly detection systems in practical environments. In this article, a novel framework, Masked Temporal Graph Generative Adversarial Network (MTG-GAN), is proposed for both conventional and cross-domain log anomaly detection. The model enhances the detection capability for emerging abnormal patterns in system log data by introducing an adaptive masking mechanism that combines generative adversarial networks with graph contrastive learning. Additionally, MTG-GAN reduces dependency on specific data distribution and improves model generalization by using diffused graph adjacency information deriving from temporal relevance of event sequence, which can be conducive to improve cross-domain detection performance. Experimental results demonstrate that MTG-GAN outperforms existing methods on multiple real-world datasets in both conventional and cross-domain log anomaly detection.

Index Terms—Log Anomaly Detection, Generative Adversarial Networks (GANs), Temporal Data Analysis

I. INTRODUCTION

THE widespread adoption of modern distributed systems, cloud computing platforms, and industrial control systems has increased the importance of system log data for anomaly detection and system monitoring. These logs capture detailed system operations, containing extensive temporal information and interaction patterns [1], [2]. Statistics show that a large-scale distributed system can generate terabytes of log data daily, encompassing millions of event records [3]. However, the high dimensionality, unstructured nature, and heterogeneity of log data across different systems present significant challenges for anomaly detection [4], [5], [6]. In reality, there are various applications

of cross-domain log anomaly detection. For example, in 5G mobile base station management, a group of base stations that are geographically adjacent and have close mutual influence are usually treated as a management cluster. When using machine learning based anomaly detection models, different clusters need to be trained separately due to significant differences in size, external environment, and diverse monitoring indicators, resulting in significant maintenance costs. From a practical perspective, an effective cross-domain log anomaly detection system can significantly reduce the operational costs, as it eliminates the need for system-specific model retraining and extensive labeled data collection when deploying to new environments. Although recent deep learning methods have achieved breakthroughs in log anomaly detection, significant limitations remain in terms of generalization capability, dynamic dependency modeling, and interpretability, constraining their practical deployment [7], [8], [9].

In practical applications, log data distribution is highly dependent on specific system environments, making cross-system model migration a fundamental challenge [10], [11]. This dependency manifests at three levels: first, significant differences in log formats and event types across systems result in feature space inconsistency; second, the diverse manifestations of anomaly patterns across different systems make single detection strategies inadequate; and third, system behavior patterns dynamically change with environmental and load variations, rendering static models ineffective [12], [13]. For instance, research in Google's production environment shows that identical anomaly detection models can exhibit performance variations 20%-30% across different server clusters [14]. This lack of cross-system generalization increases model deployment and maintenance costs and significantly impacts the reliability and practicality of anomaly detection systems [15].

Additionally, the complex temporal dependencies and dynamic evolutionary characteristics among events in log data present a second core challenge for anomaly detection [16]. Traditional methods tend to employ static sequence models or fixed adjacency matrices to represent event relationships [17], presenting three critical issues: first, they neglect long-term dependencies between events, resulting in insufficient detection capability for complex anomaly patterns [18]; second, static modeling methods cannot adapt to dynamic changes in system behavior, particularly during load fluctuations or configuration updates [16], [19]; finally,

This research is supported by the National Natural Science Foundation of China (Nos: 62172264) (*Corresponding Author: Chao Luo*).

Chao Luo, Jing Zhang are with School of Information Science and Engineering, Shandong Normal University, Jinan 250014, China. Chao Luo is with Shandong Provincial Key Laboratory for Novel Distributed Computer Software Technology, Jinan 250014, China. Rui Shao is with the center of network optimization, China Mobile Shandong Co., Ltd, Jinan 250014, China.

existing methods struggle to effectively distinguish between regular behavioral changes and genuine anomalous events, leading to high false-positive rates. For example, in OpenStack systems, certain anomalies might manifest as unusual combinations of specific event sequences or rare temporal patterns rather than individual anomalous events [20], [21]. This dynamicity and context dependency make detection methods based on static features inadequate for accurately identifying complex anomaly patterns.

Challenges lie in these two issues' interconnected and progressive nature: enhancing cross-system generalization requires models to understand the essential characteristics of event sequences across different systems, which depends on accurate modeling of temporal dependencies. Simultaneously, the quality of temporal feature modeling directly influences the model's adaptability to new systems. This complex interaction demands solutions to both challenges while achieving interpretability in anomaly detection results to support system maintenance personnel's decision-making processes [22].

This paper introduces MTG-GAN, a novel approach to cross-domain system log anomaly detection leveraging masked temporal graph generative adversarial networks. MTG-GAN addresses the critical challenges of generalization and accurate temporal dependency modeling in diverse system environments. By incorporating an adaptive masking mechanism into the training process, MTG-GAN learns robust representations by selectively masking portions of the adjacency matrix, preventing over-reliance on system-specific data patterns. This masking strategy, coupled with a generative adversarial network (GAN) architecture, empowers the model to generate diffused adjacency matrices, significantly improving robustness against distant data distributions. Further enhancing performance, a spectral loss function preserves global topological structure during adjacency matrix representation learning, enabling the capture of both short-term anomalies and long-term dependencies. Finally, a feature perturbation contrastive loss facilitates anomaly reasoning by comparing feature changes before and after masking, adding interpretability to the detection process and contributing to a more transparent understanding of anomaly causation. The main contributions are:

- Design of a diffused graph generation module based on generative adversarial networks. The generator enhances the model's capability to handle emerging anomaly patterns by simulating complex relationships between log events, which can efficiently detect known anomaly patterns and effectively handle unknown cases.
- Proposal of a dynamical masking mechanism to enhance generalization capability. By adaptively masking portions of adjacency matrix information, the model reduces dependency on specific data patterns during training, improving its transfer capability across different datasets. This mechanism effectively mitigates performance degradation caused by data distribution differences in cross-domain environments.

- Integration of spectral loss and feature perturbation contrastive loss to improve detection accuracy and interpretability. Spectral loss ensures the preservation of global topological structure, enhancing the model's ability to capture long-term dependencies; feature perturbation contrastive loss provides preliminary reasoning capability for anomaly detection through analysis of feature changes before and after masking, enhancing the transparency and interpretability of detection results.

II. RELATED WORK

A. Conventional Log Anomaly Detection

Early research on log anomaly detection relied heavily on rule-based and statistical approaches. Principal component analysis (PCA), as introduced by Xu et al. [23], pioneered the use of dimensionality reduction to identify anomalies in large-scale structured log data, marking a significant advancement in automated anomaly detection. Similarly, the clustering-based LogCluster algorithm, developed by Vaarandi et al., facilitated pattern recognition in large-scale textual log datasets, offering a straightforward yet effective means of detecting anomalous events [24]. The introduction of deep learning has significantly expanded the capabilities of log anomaly detection. DeepLog, proposed by Du et al., utilized Recurrent Neural Networks (RNNs) to predict future log events, enabling real-time anomaly detection in sequential data [1]. Expanding this approach, Meng et al. developed LogAnomaly, which incorporates semantic embeddings to detect sequential and quantitative anomalies, thereby improving robustness to unseen log patterns [25]. LogRobust, proposed by Zhang et al., addressed data instability issues with Bi-LSTM and attention mechanisms, further enhancing detection reliability under dynamic log environments [26]. Graph-based methods have emerged as a robust alternative for capturing complex interdependencies between log events. Yan et al. introduced temporal event graphs, leveraging graph-based link prediction for anomaly detection in dynamic systems [27]. Similarly, Velickovic et al.'s Deep Graph Infomax framework established a strong foundation for contrastive representation learning on graphs, later adapted for log anomaly detection [28]. Hou et al.'s GraphMAE further innovated with a self-supervised masked graph autoencoder to reconstruct graph features, addressing both local and global structure consistency [29]. Self-supervised learning techniques have also contributed to advancements in log anomaly detection. LogFiT, by Almodovar et al., demonstrated the effectiveness of fine-tuned language models for log analysis without the need for labeled data [30]. Qi et al. proposed LogEncoder, integrating contrastive representation learning and one-class classification to effectively distinguish between normal and anomalous sequences [31].

Despite significant progress, existing methods struggle to balance local feature extraction and global structural consistency, particularly in dynamic and heterogeneous

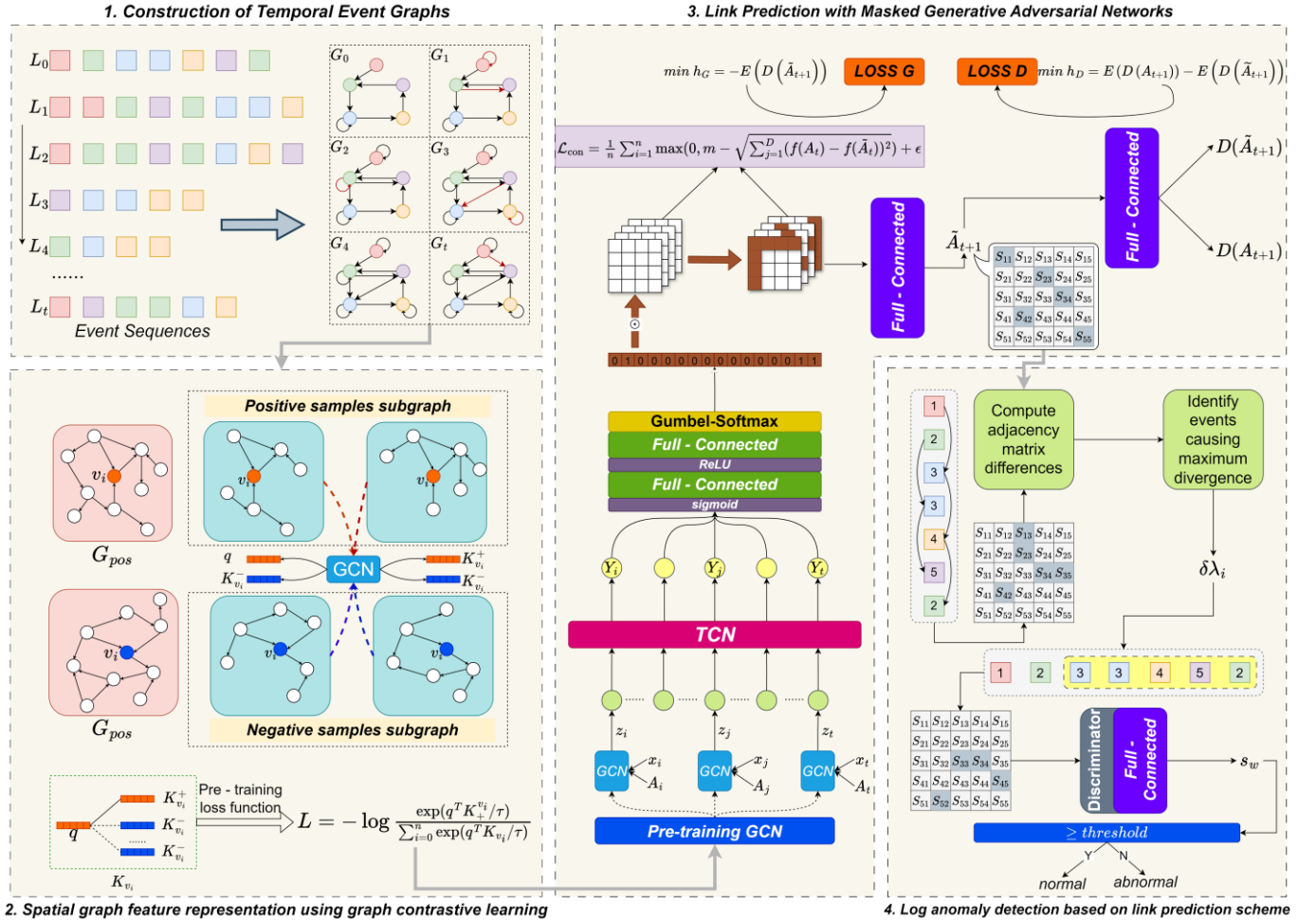


Fig. 1 Overall Architecture of MTG-GAN

environments. Hence, the above methods have generally not been tested in cross domain applications. Our proposed model, MTG-GAN, addresses these challenges by integrating graph contrastive learning with generative adversarial networks. By incorporating adaptive masking and spectral loss mechanisms, MTG-GAN effectively captures both temporal dependencies and global structure, improving anomaly detection accuracy and robustness across diverse datasets.

B. Cross-domain Log Anomaly Detection

Cross-domain anomaly detection tackles the cold-start problem of insufficient labeled data in newly deployed systems. Han et al.'s LogTAD employed adversarial domain adaptation to align feature distributions across domains, enabling models to adapt without labeled target data [32]. Similarly, LogTransfer by Chen et al. leveraged transfer learning to align semantic similarities while mitigating syntactic differences, demonstrating its utility in cross-system applications [33]. Meta-learning has emerged as a powerful technique for enhancing model generalization in cross-domain scenarios. MetaLog, introduced by Zhang et al., employed global semantic embeddings and meta-learning frameworks to achieve near-supervised performance with minimal labeled data [34]. Temporal dependencies play a pivotal role in cross-domain log anomaly detection. LogETA, proposed by Gong et

al., introduced time-aware self-attention and inter-class boundary optimization to preserve temporal relationships and align domain distributions effectively [35]. Wang et al.'s LogGT highlighted the challenges of aligning heterogeneous graph features while ensuring robust transfer learning. By combining graph-based feature extraction with domain adaptation, LogGT advanced the state-of-the-art (SOTA) in feature alignment for cross-domain tasks [36].

Current cross-domain methods often fail to address large distributional shifts and maintain temporal dependencies, limiting their accuracy and robustness. MTG-GAN overcomes these limitations by employing a masked temporal graph GAN framework. Its adaptive masking mechanism reduces domain dependency, while a generative approach ensures effective modeling of complex temporal relationships. These innovations make MTG-GAN particularly adept at handling domain shifts and enhancing anomaly detection in diverse and dynamic environments.

III. PROPOSED METHOD

The two challenges of cross-domain log anomaly detection are capturing dynamic time dependencies and cross domain generality. Simply put, the log formats and event patterns of different systems vary greatly. Traditional models are not

adapted to different systems, and the event associations in the logs will change over time. The core idea of MTG-GAN is straightforward: first, turn discrete log events into a dynamic event graphs, and then use adaptive masking as filtering mechanism to remove the "noise" unique to each system, focusing on learning event patterns that are common to different systems; Subsequently, by combining generative adversarial networks and temporal modeling capabilities, it has the ability to understand the long-term correlations of events, flexibly adapt to new systems, and distinguish between local event anomalies or overall pattern problems.

Specifically, MTG-GAN is designed to confront the dual challenges inherent in system log analysis: capturing complex temporal dependencies and emerging anomaly patterns, and ensuring robust generalization across diverse system environment. The efficacy of MTG-GAN stems from the seamless integration of several specialized components, each chosen for its specific contribution towards these goals: temporal graph representation, graph contrastive learning, adaptive masking mechanisms embedded within a generative adversarial network, and temporal convolutional networks, as illustrated in Fig. 1.

The process begins with standard preprocessing where raw logs are parsed by using Drain and partitioned into structured event sequences. Crucially, the temporal event graph construction module then transforms these linear sequences into dynamic graph representations. This structural conversion is foundational, moving beyond simple sequential modeling to explicitly encode both the temporal progression and the intricate relational dependencies between log events, thereby establishing a richer substrate for tackling capturing complex temporal dependencies and emerging anomaly patterns. To extract meaningful insights from these structures, we employ graph contrastive learning (GCL) for pre-training the graph encoder. The objective here is not merely features extraction but learning robust and discriminative node embeddings that capture essential topological information. By maximizing mutual information between augmented views of the graph, GCL yields representations that are more resilient to noise and minor structural variations, enhancing the basis for subsequent temporal analysis and inherently improving feature generalizability across different graph instances contributing to ensure robust generalization across diverse system environment.

At the core of MTG-GAN resides the mask-based generative adversarial network (GAN), a module strategically designed to synergistically address both core challenges. The adaptive masking mechanism plays a pivotal role in overcoming cross-system generalization barriers by actively suppressing domain-specific spurious correlations. By dynamically and selectively obscuring parts of the input graph's adjacency matrix during training, it prevents the model from overfitting to potentially spurious, domain-specific correlations. This forces the generator and discriminator to learn based on more fundamental, transferable structural and temporal patterns, significantly enhancing cross-domain generalization. Concurrently, the generative aspect of the GAN specifically tackles the critical task of temporal dependency modeling and anomaly pattern capture. The generator learns to predict the subsequent event graph state,

thereby modeling the system's dynamic evolution and capturing temporal dependencies. This predictive capability, refined through adversarial training against the discriminator, also endows the model with the potential to simulate complex event interrelations and anticipate emerging or unseen anomaly patterns. Furthermore, the adversarial process encourages the generation of realistic yet potentially diffused adjacency information, which further bolsters robustness against the distributional shift characteristic of cross-domain scenarios—a fundamental requirement for maintaining generalizability across heterogeneous systems. Finally, recognizing that many anomalies unfold over extended durations, a Temporal Convolutional Network (TCN) is incorporated. Operating on the sequence of graph embeddings generated over time, the TCN is specifically designed with causal convolutions and a large receptive field to explicitly capture long-range temporal dependencies—a critical capability often lacking in static or short-memory models. This architectural design directly empowers the framework to overcome the inherent limitations of conventional approaches in modeling systemic behavioral evolution and detecting multi-scale anomalies.

A. Temporal Event Graph Construction

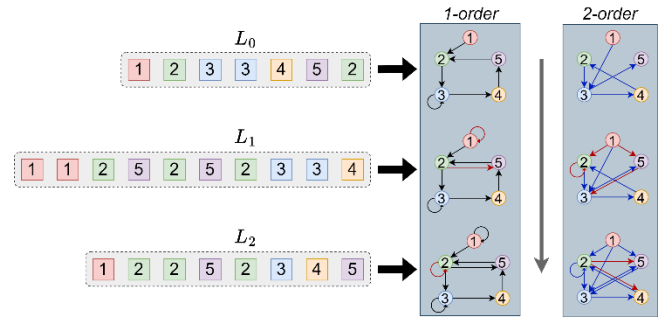


Fig. 2. Construction Process of Temporal Event Graphs

After log parsing, the parsed event templates are divided into sequences. Event sequences in log data are typically partitioned along temporal or session dimensions to capture temporal dependencies and causal relationships between events. Common partitioning methods include fixed windows, sliding windows, and session windows. This partitioning transform raw unstructured log data into event sequences that reflect temporal order or logical associations.

Constructing temporal event graphs is essential for capturing temporal dependencies in log data. We adopt an incremental approach to construct temporal event graphs to better model temporal dependencies between logs and events. Given a set of event sequences $\mathcal{L} = L_0, L_1, \dots, L_t$, where each sequence $L_i = e_0^i, e_1^i, \dots, e_T^i$ represents the i -th event sequence and e_j^i denotes the j -th event, we convert these sequences into temporal event graph sequences $\mathcal{G} = G_0, G_1, \dots, G_{\bar{t}}$, where $G_u = (V, E_u)$ with V as the node set and E_u as the edge set. The general process of transforming event sequences into corresponding temporal graph representations is illustrated in Fig. 2. By introducing a control parameter k , longer-distance event correlations can be effectively captured. The construction of temporal event graphs considers both the temporal sequence of events and multi-order adjacency

relationships, facilitating a better understanding of complex associations between log events. This approach effectively represents the temporal features of system logs in high-dimensional space, thereby improving anomaly detection accuracy.

During data processing, each log event is mapped to a node, and edges are added based on temporal relationships between events. This process ensures that the input data is already in graph form when it enters the model, facilitating subsequent graph convolutional network processing.

B. Hierarchical Learning Mechanism for Spatiotemporal Domain Feature Representation

This study proposes a hierarchical feature representation framework based on graph contrastive learning and temporal convolutional networks to capture spatiotemporal features of log events. This framework enables high-dimensional representations of log event sequences through progressive feature learning in spatial and temporal domains.

In the spatial domain representation stage, we first construct adjacency matrix sequences $\mathcal{A} = A_0, A_1, \dots, A_t$ to capture temporal relationships between events, along with corresponding feature matrix sequences $\mathcal{X} = X_0, X_1, \dots, X_t$. We use Graph Convolutional Networks (GCN) as encoders for feature embedding, formally defined as:

$$Z_i = \text{GCN}(X_i, A_i) = \sigma \left(\tilde{D}_i^{-\frac{1}{2}} \tilde{A}_i \tilde{D}_i^{-\frac{1}{2}} X_i W \right) \quad (1)$$

where $\tilde{A}_i = A_i + I_N$ is the adjacency matrix with self-loops, $\tilde{D}_i$ is the degree matrix, W is the learnable parameter matrix, and σ is the non-linear activation function.

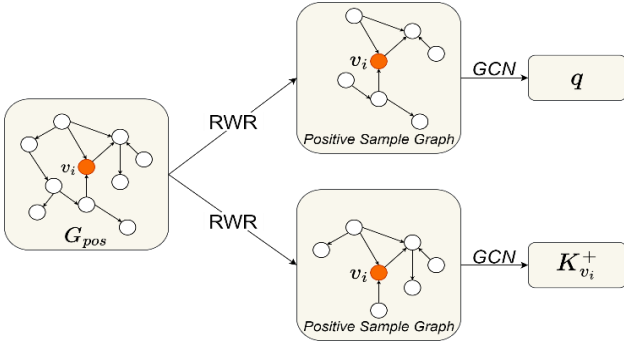


Fig. 3. Illustration of Positive Sample Construction under Graph Contrastive Learning Paradigm

We introduce a contrastive learning objective function based on InfoNCE at this stage:

$$\mathcal{L}_{\text{InfoNCE}} = -\mathbb{E}_{x, x^+} \left[\log \frac{\exp \left(\frac{s(h(x), h(x^+))}{\tau} \right)}{\sum x^- \exp \left(\frac{s(h(x), h(x^-))}{\tau} \right)} \right] \quad (2)$$

where $h(\cdot)$ represents the encoder function, $s(\cdot, \cdot)$ is the similarity measure function, τ is the temperature parameter, (x, x^+) is a positive sample pair, and x^- represents negative samples. Fig. 3 provides a conceptual illustration of positive sample pair construction within the graph contrastive learning paradigm, which typically involves creating augmented views

or sampling structurally similar subgraphs. Minimizing this loss function enables unsupervised learning of graph structural features while enhancing the model's sensitivity to topological changes. For temporal domain representation, we employ a TCN with causality constraints for sequence modeling. Given the feature sequence $Z = Z_0, Z_1, \dots, Z_t$ output by the GCN, the TCN's progressive feature extraction process is defined as:

$$Y^t = \text{TCN}(Z) = f_n \circ f_{n-1} \circ \dots \circ f_1(Z) \quad (3)$$

where f_i represents the transformation function of the i -th TCN layer. Core operations in each TCN layer include causal convolution, non-linear activation, and residual connections, with causal convolution calculated as:

$$f_i^{\text{conv}}(Z)_t = \sum_{k=0}^{K-1} w_k Z_{t-k} \quad (4)$$

where K is the convolution kernel size and w_k represents learnable kernel parameters. By stacking multiple causal convolution layers, the TCN achieves exponentially expanding receptive fields, effectively capturing long-term temporal dependencies. The introduction of residual connections alleviates the gradient vanishing problem in deep networks, mathematically expressed as:

$$f_i^{\text{residual}}(Z) = \text{ReLU}(f_i^{\text{conv}}(Z) + Z) \quad (5)$$

This hierarchical feature representation mechanism establishes a feature learning framework that progresses from local to global, from spatial to temporal domains. At the implementation level, this framework maps unstructured log sequences to structured feature spaces by constructing high-dimensional adjacency and feature matrices. The spatial domain graph contrastive learning focuses on capturing topological relationships between events, while the temporal domain convolutional network extracts temporal patterns. Together, these components provide feature representations rich in spatiotemporal information, enhancing anomaly detection.

C. Link Prediction via Masked Generative Adversarial Networks

At the heart of MTG-GAN lies an innovative link prediction framework that integrates a GAN with an adaptive masking mechanism. Its objective function is defined as:

$$\min_G \max_D \mathbb{E}_{A \sim p_{\text{data}}} [\log D(A)] + \mathbb{E}_{z \sim p_z} \left[\log \left(1 - D \left(G(z, M(A)) \right) \right) \right] \quad (6)$$

where G , D , and M represent the generator, discriminator, and masking module respectively, A is the real adjacency matrix, and z is random noise. Through adversarial generation, this framework effectively models both normal and anomalous patterns in log events, wherein the generator synthesizes graph representations approximating real adjacency matrices, while the discriminator iteratively refines its discriminative capacity through an adversarial learning process. The incorporation of an adaptive masking mechanism introduces controlled stochastic perturbations into the input data, consequently enhancing the robustness of both the generator and discriminator while significantly improving the model's generalization capability towards unseen anomaly patterns. In data processing, the adjacency matrix undergoes adaptive masking before being fed into the generator, ensuring model

robustness to partial information loss while making generated features more diverse, thereby improving anomaly detection effectiveness.

1) Generation Mechanism Based on Selective Masking

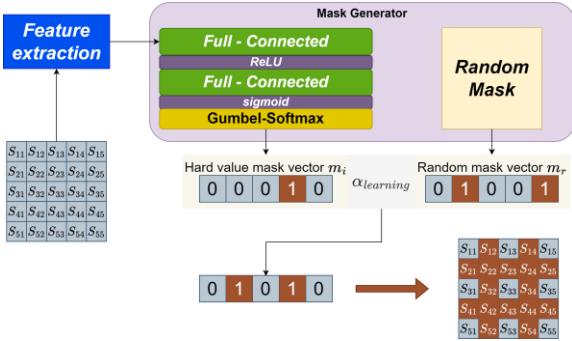


Fig. 4. Adaptive Masking Mechanism combining learned (Gumbel-SoftMax based) and random masks.

This study introduces a novel graph generation mechanism leveraging adaptive selective masking to address the inherent challenges in system log anomaly detection, where graph-structured data exhibits complex temporal dependencies and structural biases that significantly complicate feature learning. Although conventional random masking approaches provide limited enhancement to model robustness, their inherent inability to preserve structural information and temporal features in graph data significantly impairs the model's capacity to capture anomalous patterns effectively. Based on this observation, we designed a dual masking mechanism with dynamic weight adjustment, achieving adaptive capture of key structural features through the synergistic effect of learned and random masking. The overall adaptive masking process, illustrating how the generated learned mask (m_{learning}) is combined with a random mask (m_r) using the adaptive weight $\alpha_{\text{adv}}(t)$, is depicted in Fig. 4.

In the design of selective learning masks, we constructed a differentiable mask generation module based on Gumbel-SoftMax. Compared to discrete Bernoulli sampling, Gumbel-SoftMax provides a differentiable discrete approximation, enabling end-to-end training of the mask generation process. Given input feature tensor $X \in \mathbb{R}^{B \times W \times N \times F}$ (where B, W, N, F represent batch size, time window length, number of nodes, and feature dimension respectively), the mask generation process can be formalized as a two-stage feature transformation:

$$h_l = \text{LayerNorm}(\text{ReLU}(W_1 X + b_1))$$

$$m_{\text{learning}} = \text{GumbelSoftmax}\left(\frac{W_2 h_l + b_2}{\sqrt{d_k}}, \tau\right) \quad (7)$$

where $W_1 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$, $W_2 \in \mathbb{R}^{d_{\text{ff}} \times N}$ are learnable transformation matrices, d_k is the feature dimension scaling factor, and τ is the temperature parameter. The advantages of this design are: first, normalizing feature distributions through LayerNorm reduces the impact of Internal Covariate Shift; second, introducing the scaling factor $\sqrt{d_k}$ effectively mitigates the gradient vanishing problem caused by increasing feature dimensions. To address the dynamic adjustment of

masking strategies during training, we propose an adaptive weight function with nonlinear growth characteristics:

$$\alpha_{\text{adv}}(t) = \alpha_{\text{start}} + \left(\frac{t}{T_{\text{pretrain}}}\right)^{\gamma} (\alpha_{\text{end}} - \alpha_{\text{start}}) \cdot \exp\left(-\beta \left|\frac{t}{T_{\text{pretrain}}} - 0.5\right|\right) \quad (8)$$

This function is designed based on the following considerations: in the early stages of training, when the model's understanding of data distribution is rudimentary, more random exploration is needed; as training progresses, the model gradually masters the inherent patterns of the data, at which point the proportion of learned masking should increase to capture key features. The introduction of the exponential decay term provides a smooth transition period during mid-training, avoiding training instability caused by abrupt weight changes.

We employ a contrastive loss function, $\mathcal{L}_{\text{con}}$, to encourage the model to learn representations that are sensitive to the structural changes introduced by masking. This loss function is defined as:

$$\mathcal{L}_{\text{con}} = \frac{1}{n} \sum_{i=1}^n \left[\max\left(0, m - \sqrt{\sum_{j=1}^D (f(A_t)_j - f(\tilde{A}_t)_j)^2}\right) \right] + \epsilon \quad (9)$$

where n is the number of samples in a batch, D is the dimensionality of the feature representations, m is a margin parameter that controls the separation between positive (similar) and negative (dissimilar) pairs, and ϵ is a small constant for numerical stability. Minimizing this loss encourages the feature representations of the original and masked adjacency matrices to be close, within a margin m . This effectively forces the model to learn representations that are invariant to the masked elements, thereby focusing on the core structural information that remains. This process allows the generator to learn to better distinguish anomalies by implicitly learning the characteristics of the self-supervised anomalies introduced by the masking process, ultimately enhancing the model's anomaly detection capabilities.

For preserving global topological structure of graphs, simple node-level or edge-level reconstruction losses often struggle to capture high-order structural features. Based on this understanding, we introduce a generalized spectral loss function based on spectral graph theory:

$$\mathcal{L}_{\text{spectral}} = |\Phi(L_{\text{real}}) - \Phi(L_{\text{pred}})|^2 F^2 + \lambda \text{tr}\left(\Phi(L_{\text{real}})^T \Phi(L_{\text{pred}})\right) \quad (10)$$

The design of this loss function is based on the following theoretical foundations: first, the eigen spectrum of the Laplacian matrix contains essential structural information of the graph, such as connectivity and clustering characteristics; second, through the introduction of the spectral projection operator $\Phi(\cdot)$, we can better measure structural differences between graphs in the frequency domain; finally, the introduction of the trace term provides a soft constraint, ensuring alignment of eigenvector spaces, which is particularly important for preserving local structural features of the graph.

2) Discriminator Design and Optimization

Building upon the aforementioned generation framework, we propose a structurally-aware discriminator architecture that incorporates sophisticated topological learning capabilities. Our discriminator extends beyond conventional GAN architectures by not only differentiating between authentic and synthesized graph structures but also facilitating the generator's learning process through dual mechanisms: feature uniformity constraints and spectral domain consistency preservation, thereby promoting the generation of more realistic anomaly patterns. To address the inherent high dimensionality and sparsity of graph-structured data, we implement a multi-layer discrimination framework utilizing feed-forward neural networks as the architectural foundation:

$$D(A) = \text{FFN}(\text{vec}(A)) \quad (11)$$

where A is the input adjacency matrix, $\text{vec}(\cdot)$ represents the matrix flattening operation, and FFN denotes a feed-forward neural network with nonlinear activation functions. Through this design, the discriminator can effectively capture global features of graph structures. To enhance the model's discriminative capability, we adopt a loss function based on Wasserstein distance:

$$\mathcal{LD} = \mathbb{E}[D(A_{\text{real}})] - \mathbb{E}[D(A_{\text{fake}})] \quad (12)$$

where A_{real} and A_{fake} represent real and generated adjacency matrices respectively. This formulation of the loss function yields dual advantages: it ensures gradient stability during training while providing a more robust metric for quantifying the distributional divergence between real and generated samples. During training, the discriminator's parameter updates follow the optimization objective:

$$\min_D \mathcal{LD} + \lambda \mathcal{L}_{gp} \quad (13)$$

where $\mathcal{L}_{gp}$ is the gradient penalty term used to enforce the 1-Lipschitz constraint on the discriminator, and λ is the weight coefficient. This optimization strategy ensures that the discriminator maintains moderate discriminative ability during training, avoiding gradient vanishing or explosion problems. This streamlined discriminator architecture achieves a favorable balance between model complexity and detection efficacy. Empirical evaluations demonstrate that our design effectively steers the generator towards producing more faithful graph structure representations, consequently elevating the model's overall anomaly detection capabilities.

D. Anomaly Score Evaluation Method

We propose a novel anomaly scoring framework, depicted in Fig. 5, that integrates spectral graph theory with the generative adversarial network to enable precise detection and quantitative evaluation of diverse system anomalies by comprehensively analyzing both local event patterns and global structural changes. The scoring process begins with the pre-trained generator predicting the expected adjacency matrix S_{pred} for a given input sequence, representing the anticipated normal state.

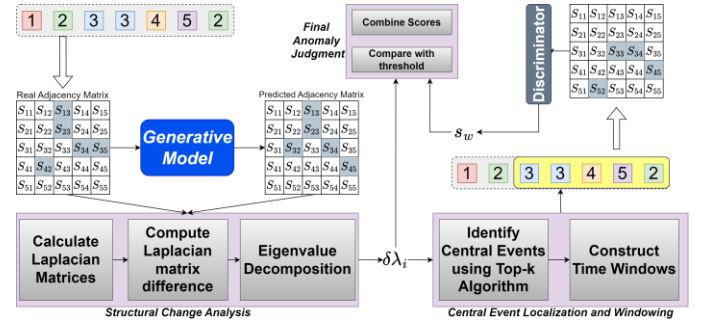


Fig. 5. Flow Chart of Anomaly Score Calculation

This prediction is then compared against the actual observed adjacency matrix (S_{real}) using a differential analysis framework based on graph Laplacians. For the pair S_{real} and S_{pred} , their corresponding Laplacian matrices are computed as:

$$\begin{aligned} L_{\text{real}} &= D_{\text{real}} - S_{\text{real}} \\ L_{\text{pred}} &= D_{\text{pred}} - S_{\text{pred}} \end{aligned} \quad (14)$$

where D_{real} and D_{pred} are the degree matrices. By calculating the Laplacian difference $\Delta L = L_{\text{real}} - L_{\text{pred}}$ and performing eigenvalue decomposition, we obtain the eigenvalue difference sequence $\delta\lambda_{ii=1}^n$. Crucially, as the Laplacian spectrum encapsulates the graph's global topological properties, such as connectivity and clustering characteristics, the magnitude of the L2 norm of this difference, $\|\delta\lambda\|_2 = \|\delta\lambda_{ii=1}^n\|_2$, serves as a potent indicator of global structural anomalies. Significant deviations suggest substantial shifts in the overall system behavior and event interaction patterns, often associated with major faults or configuration changes. The derived eigenvalues and their associated eigenvectors also aid in identifying pivotal event nodes exhibiting maximum structural impact, enabling targeted analysis.

Based on these central events identified via an improved top-k algorithm applied to the spectral analysis results, time windows are constructed. Event subsequences within these windows are converted into subsequence adjacency matrices and fed into the discriminator network. The discriminator, trained to distinguish real (normal) from generated graph structures, evaluates these subsequences, outputting an anomaly score S_w . This score, S_w , reflects the discriminator's assessment of the subsequence's consistency with learned normal local event patterns. Since the discriminator implicitly learns valid short-term sequences and contextual relationships during adversarial training, a high S_w points towards local pattern anomalies, such as the occurrence of rare events, incorrect event ordering within a short timeframe, or deviations from expected contextual behavior within the window.

The final comprehensive anomaly score S_{total} is then calculated as a weighted sum:

$$S_{\text{total}} = \beta \cdot S_w + (1 - \beta) \cdot \|\delta\lambda_{ii=1}^n\|_2 \quad (15)$$

where β is a weight parameter balancing the contributions from the two complementary perspectives. The term S_w captures deviations in local sequence patterns within specific, potentially problematic time windows, while the term $\|\delta\lambda_{ii=1}^n\|_2$ quantifies shifts in the global graph structure, reflecting large-scale behavioral changes. This sophisticated multi-dimensional scoring mechanism synthesizes both local

and global viewpoints, yielding a more comprehensive and robust anomaly detection framework capable of identifying local pattern anomalies (indicated primarily by high S_w), global structural anomalies (indicated primarily by high $||\delta\lambda||_2$), and hybrid scenarios exhibiting characteristics of both. This synthesis significantly enhances detection accuracy and reliability across diverse anomaly types. Anomaly determination is subsequently accomplished through a dynamic thresholding mechanism that adaptively adjusts based on historical data distributions and operational contexts, ensuring contextually appropriate detection criteria across varying scenarios. For instance, during periods of elevated system load, the framework dynamically modulates its thresholds to accommodate increased frequencies of anomalous yet potentially benign event sequences, thereby mitigating false positive rates. This adaptive thresholding strategy substantially enhances the model's operational efficacy and reliability in production environments, particularly in systems with varying load patterns and operational characteristics.

IV. EXPERIMENTS

A. Datasets

- 1) HDFS dataset [37]: This dataset comprises log data generated from executing Hadoop-based map-reduce jobs. It contains 11,175,629 log messages, with each message containing a block ID. Using session windows for sequence partitioning, parsing yielded 558,221 normal sequences, 16,838 anomalous sequences, and 29 event templates.
- 2) BGL dataset [37]: This dataset consists of log data generated by the BlueGene/L supercomputer system. It contains 4,747,963 log messages, partitioned using a timestamp-based sliding window approach. The window size is 6 hours with a 1-hour step size. Parsing yielded 2,969 normal sequences, 1,723 anomalous sequences, and 379 event templates.
- 3) OpenStack dataset [37]: This dataset contains log data generated on CloudLab. It comprises 207,820 log messages, partitioned using a timestamp-based fixed window approach. With a window size of 1 minute, parsing yielded 1,634 normal sequences, 138 anomalous sequences, and 72 event templates.
- 4) Thunderbird dataset [37]: This dataset comes from the Thunderbird supercomputer system at Sandia National Laboratories (SNL). It contains 211,212,192 log messages, partitioned using a timestamp-based fixed window approach. With a window size of 1 hour, parsing yielded 268,055 normal sequences, 70,403 anomalous sequences, and 1,147 event templates. For anomaly detection experiments, a portion of the data was randomly sampled.
- 5) SWaT Dataset [38]: Provided by iTrust SUTD, this dataset contains multivariate time series (MTS) from a

secure water treatment testbed. Our analysis used data from $N=30$ critical sensors/actuators over 7 normal and 4 attack days. The continuous MTS was segmented using a 60s sliding window with a 10s stride into windowed sequences X_t^w . Following a graph construction methodology analogous to that presented in [39], each sequence X_t^w was subsequently organized into a temporal graph $G_t = (V, E_t)$, where V represents the nodes (sensors/actuators) and E_t captures the dynamic interactions inferred within that time window. The initial node features were derived from X_t^w .

- 6) WADI Dataset [40]: As an extension of SWaT, this dataset represents a water distribution system developed by iTrust SUTD, containing MTS data from $N = 121$ sensors and actuators over 14 days of normal operation and 2 days of anomalies. Our study utilizes $N = 75$ relevant nodes and focuses on 13 specific attack cases. The data processing methodology aligns with that of SWaT.

B. Evaluation Metrics

To measure anomaly detection performance, Precision, Recall, F1 Score, and AUC were used as evaluation metrics, defined as follows:

$$\begin{aligned} \text{Precision} &= \frac{TP}{TP + FP}, \\ \text{Recall} &= \frac{TP}{TP + FN}, \\ F1 &= \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}, \\ AUC &= \frac{\sum_{i \in \text{positive class}} \text{rank}_i - \frac{M(1+M)}{2}}{M \times N}. \end{aligned}$$

where TP represents true positives, indicating the number of correctly identified anomalous sequences; FP represents false positives, indicating the number of normal sequences incorrectly identified as anomalous; FN represents false negatives, indicating the number of anomalous sequences incorrectly identified as normal; rank_i is the rank of the i -th sample; M is the number of positive samples, and N is the number of negative samples.

C. Experiment results

MTG-GAN demonstrates exceptional anomaly detection performance across diverse benchmark datasets (TABLE I for details), spanning scenarios from traditional system logs (e.g., HDFS, BGL) to industrial control system (ICS) time series (e.g., SWaT, WADI). The model not only establishes a new SOTA on the complex BGL dataset with an F1 score of 96.31% but also achieves highly competitive F1 scores (generally ranging from 93% to 98%, depending on the dataset) across all other tested datasets.

TABLE I
COMPARISON OF DIFFERENT METHODS FOR CONVENTIONAL LOG ANOMALY DETECTION

Dataset	Method	Precision (%)	Recall (%)	F1 score (%)
HDFS	DeepLog (2017) [1]	92.98	90.09	91.51

	LogAnomaly (2019) [25]	93.46	59.20	72.49
	LogBASA (2023) [41]	99.30	98.10	98.70
	LogEncoder (2023) [31]	94.40	94.50	94.30
	LogGPT (2023) [42]	88.40	92.10	90.10
	DLAD (2023) [27]	99.50	96.40	97.93
	LogFiT (2024) [30]	99.78	90.70	95.02
	MTG-GAN	90.56	99.40	94.77
BGL	DeepLog (2017) [1]	92.31	90.91	91.60
	LogAnomaly (2019) [25]	78.03	86.70	82.14
	LogBASA (2023) [41]	95.10	92.80	93.90
	LogEncoder (2023) [31]	89.60	93.50	87.60
	LogGPT (2023) [42]	94.00	97.70	95.80
	DLAD (2023) [27]	97.40	94.20	95.77
	LogFiT (2024) [30]	94.39	85.80	89.89
	MTG-GAN	93.52	99.27	96.31
Thunderbird	DeepLog (2017) [1]	69.20	80.31	74.34
	LogAnomaly (2019) [25]	79.48	98.55	88.39
	LogBASA (2023) [41]	97.20	90.30	93.60
	LogEncoder (2023) [31]	48.30	20.10	94.70
	LogGPT (2023) [42]	97.30	100.00	98.60
	DLAD (2023) [27]	98.50	98.10	98.30
	LogFiT (2024) [30]	89.72	98.60	93.94
	MTG-GAN	94.39	99.30	96.31
OpenStack	DeepLog (2017) [1]	64.94	92.87	76.43
	LogAnomaly (2019) [25]	79.48	88.10	83.57
	ULAD (2020) [43]	92.90	94.10	93.10
	Trine (2022) [44]	80.00	93.50	88.90
	PRPCA-CS (2022) [45]	100.00	87.00	93.00
	DLAD (2023) [27]	94.00	85.60	89.60
	MTG-GAN	95.40	97.63	96.50
SWaT	GSC-MAD (2023) [46]	96.73	95.11	95.91
	CTAD (2024) [47]	97.10	74.20	84.10
	SA2 (2024) [48]	96.20	82.71	88.95
	STAGED (2024) [39]	97.28	93.72	96.47
	HSC (2025) [49]	90.80	95.80	84.70
	AnoDual (2025) [50]	96.54	75.29	84.60
	ConFlow (2025) [51]	89.06	91.38	90.26
	MTG-GAN	91.00	96.36	93.60
WADI	SA2E-AD (2024) [52]	83.37	82.95	83.16
	KeAD (2024) [53]	87.97	65.71	75.22
	CTAD (2024) [47]	79.20	34.60	48.10
	SA2 (2024) [48]	77.77	52.43	62.63
	ConFlow (2025) [51]	92.63	93.88	93.19
	AnoDual (2025) [50]	66.20	58.76	62.26
	MGRD (2025) [54]	52.00	44.90	48.20
	MTG-GAN	93.79	91.56	92.66

A core advantage lies in its consistently high recall (steadily exceeding 95% and even reaching over 99% on most datasets), which is crucial for minimizing missed detections of critical anomalies. This capability likely stems from several key aspects of the model's design: the Temporal Convolutional Network (TCN) effectively captures long-range dependencies potentially indicative of anomalies, while the graph-based representation learning (combining GCL pre-training and spectral loss) enables sensitive identification of structural deviations in event interaction patterns, even subtle ones.

Concurrently, we observed that precision can fluctuate depending on dataset characteristics, such as the number of event templates and the clarity of anomaly patterns. This suggests that the adaptive masking mechanism, while enhancing generalization and capturing emerging anomaly patterns (boosting recall), might introduce a higher false

positive rate under certain conditions (especially on noisy data or data with significant normal pattern variations), reflecting a potential trade-off between robustness and precision.

Nevertheless, the high adaptability and overall superior performance demonstrated by MTG-GAN when handling structurally diverse data from different sources robustly validate the effectiveness of its framework. Notably, its success on both traditional logs and physical system time series data highlights the flexibility of the temporal graph representation and the potential of the GAN in modeling diverse underlying data distributions. Collectively, these results validate MTG-GAN as a powerful and versatile anomaly detection solution, particularly suitable for practical application scenarios requiring high detection rates and cross-system deployment capabilities.

TABLE II
COMPARISON OF DIFFERENT METHODS FOR CROSS-DOMAIN LOG ANOMALY DETECTION

	Method	F1 (Source)	AUC (Source)	F1 (Target)	AUC (Target)
BGL → TB	PCA w/ TB	0.322	0.587	0.731	0.776
	PCA w/o TB	0.642	0.816	0.558	0.504
	LogCluster w/ TB	0.530	0.746	0.677	0.716
	LogCluster w/o TB	0.713	0.829	0.559	0.504
	DeepLog w/ TB	0.662	0.854	0.590	0.619
	DeepLog w/o TB	0.678	0.867	0.556	0.500
	DeepSVDD w/ TB	0.499	0.725	0.567	0.616
	DeepSVDD w/o TB	0.566	0.789	0.577	0.646
	LogTransfer(Supervised)(2020) [33]	0.971	0.972	0.792	0.828
	LogTAD(Unsupervised)(2021) [32]	0.926	0.964	0.792	0.828
	LogGT(Unsupervised)(2024) [36]	0.995	0.964	0.919	-
	LogETA(Supervised)(2024) [35]	0.957	0.959	0.947	0.963
	MTG-GAN(Unsupervised)	0.963	0.921	0.951	0.937
TB → BGL	PCA w/ BGL	0.789	0.798	0.577	0.773
	PCA w/o BGL	0.760	0.779	0.229	0.658
	LogCluster w/ BGL	0.708	0.688	0.697	0.886
	LogCluster w/o BGL	0.724	0.716	0.223	0.500
	DeepLog w/ BGL	0.687	0.701	0.527	0.843
	DeepLog w/o BGL	0.660	0.677	0.223	0.500
	DeepSVDD w/ BGL	0.660	0.699	0.196	0.537
	DeepSVDD w/o BGL	0.794	0.808	0.195	0.497
	LogTransfer(Supervised)(2020) [33]	0.995	0.995	0.788	0.833
	LogTAD(Unsupervised)(2021) [32]	0.788	0.797	0.845	0.909
	LogGT(Unsupervised)(2024) [36]	0.985	0.991	0.798	-
	LogETA(Supervised)(2024) [35]	0.997	0.996	0.880	0.898
	MTG-GAN(Unsupervised)	0.963	0.976	0.892	0.899
BGL → SWaT	MTG-GAN	0.963	0.921	0.818	0.766
BGL → WADI		0.963	0.921	0.795	0.709
SWaT → WADI		0.936	0.882	0.768	0.738
WADI → SWaT		0.926	0.876	0.791	0.754

D. Cross-Domain Generalization Evaluation

In the log↔log transfer scenarios (BGL↔TB), MTG-GAN demonstrates robust performance. For the BGL→TB transfer, it achieves a target F1 score of 0.951 and AUC of 0.937, outperforming several established baseline and even specialized unsupervised transfer methods like LogTAD (Target F1=0.792) and LogGT (Target F1=0.919), while being competitive with the supervised LogETA (Target F1=0.947). Similarly, in the TB→BGL scenario, MTG-GAN obtains a target F1 score of 0.892 and AUC of 0.899, again showing strong performance compared to baselines and unsupervised methods like LogTAD (Target F1=0.845) and LogGT (Target F1=0.798). These results highlight the effectiveness of MTG-GAN's architecture in capturing transferable patterns between different system log environments. The adaptive masking mechanism likely plays a crucial role by preventing overfitting to source-specific log event patterns and forcing the model to learn more fundamental temporal and structural dependencies, which are more likely to generalize. Furthermore, the temporal convolutional network's ability to model long-range dependencies and the GAN's capability to implicitly align data distributions contribute to this strong cross-system performance.

We further investigate MTG-GAN's generalization capabilities across fundamentally different data types by conducting log↔ICS transfer experiments. Transferring from BGL (system logs) to SWaT (ICS time-series), MTG-GAN

achieves a target F1 score of 0.818 and AUC of 0.766. Similarly, for BGL→WADI, the target F1 is 0.795 and AUC is 0.709. While these scores represent a noticeable drop compared to the log↔log transfer results, they demonstrate that MTG-GAN can achieve non-trivial generalization even across highly disparate domains. A plausible explanation for the performance difference lies in the nature of the node features. In log datasets like BGL and TB, node features are derived from semantic embeddings of log templates, potentially capturing abstract event types (e.g., 'connection error', 'authentication failure') that might share underlying similarities across different software systems. In contrast, SWaT and WADI node features represent physical sensor readings or actuator states (e.g., water level, flow rate), which are inherently tied to the specific physical process and lack the direct semantic correspondence found in logs. Bridging this semantic gap between system event logs and physical process data presents a greater challenge for transfer learning. Nonetheless, the fact that MTG-GAN still yields reasonable performance suggests its graph-based structure learning and temporal modeling can capture some universal dynamics related to sequential behavior and node interactions, even when node semantics differ significantly.

Finally, we evaluate transfer performance between the two related ICS datasets, SWaT and WADI. For SWaT→WADI, the target F1 score is 0.768 (Target AUC is 0.738). For WADI→SWaT, the target F1 score is 0.791 (Target AUC is 0.754). The performance in these ICS↔ICS transfers is notably better

than the BGL→WADI transfer, likely because SWaT and WADI represent similar physical systems (water treatment and distribution), allowing for more effective knowledge transfer regarding sensor interactions and process dynamics. An interesting observation is the slight asymmetry in performance; the transfer from WADI to SWaT appears marginally easier. Given that WADI is documented as an extension of the SWaT testbed, it might encompass more complex dynamics or attack scenarios. Transferring from the more complex WADI system to the potentially simpler core SWaT system could leverage learned representations of shared fundamental dynamics more readily than transferring from SWaT to the extended WADI system, which might contain novel behaviors not seen in the source SWaT data.

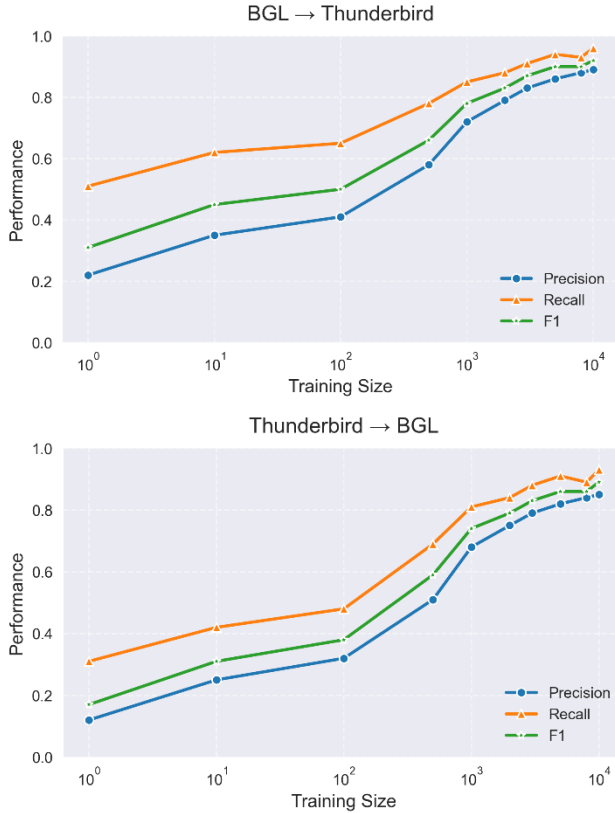


Fig. 6. Impact of Target Domain Training Sample Size on Cross-Domain Performance.

Overall, the cross-domain experiments summarized in TABLE II validate MTG-GAN's strong generalization capabilities. The adaptive masking mechanism, combined with graph contrastive learning for robust structural representations, generative adversarial training for distribution alignment, and temporal convolutional networks for capturing sequential dynamics, equips the model to effectively transfer knowledge across diverse system types and data modalities. While performance is naturally influenced by the degree of similarity between source and target domains, particularly concerning the nature of node features, MTG-GAN demonstrates significant potential for practical deployment in scenarios requiring cross-system applicability.

To analyze the impact of training sample size on model performance in depth, we conducted experiments with target

domain sample sizes ranging from 1 to 10,000 sequences in both transfer scenarios. Fig. 6 how performance variations under different sample sizes. The experiments used Precision, Recall, and F1-score as evaluation metrics, with each experimental point repeated 5 times to ensure result stability. Several key characteristics can be observed from the results: First, in the BGL→TB scenario, the model exhibited clear staged learning characteristics. With sample sizes below 500, model performance was relatively low, with precision fluctuating between 0.22-0.58, though recall maintained high levels (0.51-0.78). This performance characteristic reflects a key advantage of MTG-GAN: even with insufficient training data, the model can maintain strong detection capability for anomalous patterns through graph structures and masking mechanisms, albeit with some false positives. When sample size reached 1,000-3,000, model performance showed significant improvement, with precision increasing from 0.72 to 0.83, recall from 0.85 to 0.91, and F1-score correspondingly from 0.78 to 0.87. This rapid improvement phase indicates that as target domain data accumulates, the model can construct more reliable event graph structures, thereby improving detection accuracy. After exceeding 3,000 samples, performance improvement became gradual but maintained an upward trend, ultimately achieving 0.89 precision and 0.96 recall with 10,000 samples. In the more challenging TB→BGL scenario, due to the complexity of the Thunderbird dataset, the model showed more pronounced performance disadvantages in the small sample phase. With sample sizes below 500, precision was only 0.12-0.51, and although recall was relatively better (0.31-0.69), the overall F1-score remained low (0.17-0.59). This performance characteristic confirms our theoretical analysis: when source domain data distribution is more complex, the model requires more target domain samples to achieve effective knowledge transfer. As sample size increased, model performance showed a similar but steeper improvement curve. In the 1,000~3,000 sample range, performance rapidly improved, with precision increasing from 0.68 to 0.79 and recall from 0.81 to 0.88. Notably, even in this more challenging scenario, the model's recall consistently maintained high levels, verifying MTG-GAN's robustness in capturing anomalous patterns. Comparing performance curves from both transfer scenarios yields several key findings: First, MTG-GAN demonstrated high recall advantages in both scenarios, benefiting from effective anomaly pattern capture through graph structure learning and key feature retention through the masking mechanism. Second, model performance showed an inflection point around 1,000 samples, reflecting the minimum data volume needed to construct reliable graph structures. Third, slight performance fluctuations in the large sample region (5,000-10,000) reflect the model's sensitivity to distribution differences across different batches, a characteristic that helps the model continuously adapt to target domain distribution changes. Finally, performance differences between the two scenarios confirm the impact of dataset complexity on transfer learning and validate our data configuration design considerations. These experimental results indicate that while MTG-GAN requires relatively more target domain samples to achieve optimal performance, its unique model architecture maintains good anomaly detection

capability even with insufficient samples, particularly in terms of recall. This characteristic makes MTG-GAN particularly suitable for practical application scenarios where log data can be continuously collected, allowing the model to gradually improve performance while maintaining high detection rates for anomalous events.

E. Ablation Study

An ablation study (illustrated in Fig. 7) quantified the contributions of MTG-GAN's core components to model performance. Removing the adaptive masking mechanism had the most significant impact on performance, especially on the BGL and Thunderbird datasets featuring more event templates and complex structures, causing a sharp drop in F1 score by 5.06 percentage points on Thunderbird, for instance. This directly highlights its critical role in enhancing generalization capability by mitigating overfitting to specific data patterns. Similarly, removing the Graph Contrastive Learning (GCL) pre-training also resulted in substantial performance degradation on BGL and Thunderbird (3.77 percentage point decrease in F1 on Thunderbird), demonstrating the crucial importance of pre-learning robust topological embeddings in complex graphs for subsequent tasks.

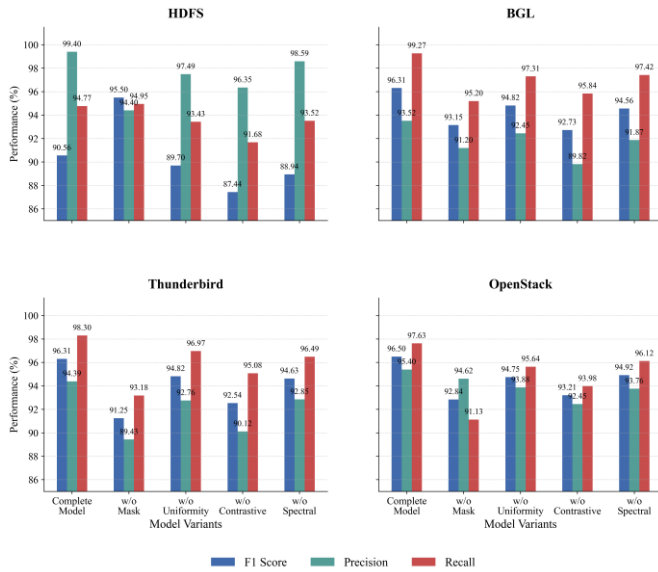


Fig. 7. Comparison of Ablation Study Results across Different Datasets

The removal of the spectral loss, impacting global structure preservation, was more pronounced on complex datasets, leading to an F1 score decrease of approximately 1.7 percentage points on BGL and Thunderbird, compared to a smaller impact on the relatively simpler HDFS dataset (around 1.25 percentage points). This indicates its value in capturing large-scale behavioral changes is particularly evident in complex systems. The uniformity loss, on the other hand, provided consistent performance improvements across all datasets (boosting F1 scores by about 1.3-1.8 percentage points), confirming its general benefit in improving feature space distribution and enhancing overall model stability. Collectively, these results validate the necessity of each MTG-GAN component for achieving high performance and their synergistic effects. They also indicate that the importance of

different components is indeed modulated by the intrinsic characteristics of the dataset, such as structural complexity.

F. Complexity Analysis and Implementation Details

TABLE III
MTG-GAN Resource Utilization during Training and Inference (Measured on BGL Dataset)

Stage	Average Memory Utilization	Average CPU Utilization	Average VRAM Utilization	Average GPU Utilization
Train	5.4 GB	12.15%	3.8 GB	22.8%
Inference	2.3 GB	10.08%	200.0 MB	12.5%

The model's training complexity is primarily composed of several core components. Specifically, the GCN encoder contributes a complexity of $O(L \times |E| \times d)$ for its L -layer forward propagation when processing a graph containing $|E|$ edges (where $|E|$ denotes the number of edges in a temporal graph instance) and node features of dimension d . Subsequently, the Temporal Convolutional Network (TCN) module, which handles sequences of length W across L_{TCN} layers, exhibits a complexity of $O(L_{TCN} \times W \times k \times d_{in} \times d_{ch})$, dependent on the kernel size k , as well as the input and channel dimensions d_{in} and d_{ch} . Furthermore, the complexity of the adaptive masking module is mainly determined by its internal feed-forward network layers, approximating $O(W \times N \times F \times d_{ff})$, and scales with the sequence length W , the number of nodes/templates N , the input feature dimension F , and the feed-forward dimension d_{ff} . Finally, the spectral loss computation part, involving N nodes and k' eigenvectors, adds a main computational cost per batch of approximately $O(N \times (k')^2)$. Synthesizing the computational costs of these components, the overall training time complexity of the model is the sum of these individual terms. Among these, the processing of the graph structure by the GCN, specifically the $O(L \times |E| \times d)$ term, typically represents the key factor influencing the model's scalability and constitutes the main computational bottleneck.

This quadratic dependency on the number of event templates underscores a critical consideration for the model's scalability in real-world deployments facing diverse and numerous log types. All experiments reported herein were conducted on a server platform equipped with an Intel® Xeon® Silver 4214R CPU operating at 2.40GHz, complemented by 256 GB of system RAM, and running Ubuntu 20.04.6 LTS. For computational acceleration, a single NVIDIA GeForce RTX 3090 GPU possessing 24 GB of VRAM was utilized. The core software environment was managed using Python 3.9.21. The implementation relied significantly on PyTorch 2.3.1 as the primary deep learning framework, interfaced with CUDA Toolkit 11.8. Graph-specific operations and sequence modeling were facilitated by the Deep Graph Library (DGL) version 2.3.0.

The average computational resource utilization for MTG-GAN, measured on the BGL dataset, is detailed in TABLE III. The training phase utilized 5.4 GB RAM, 12.15% CPU, 3.8 GB VRAM, and 22.8% GPU on average. In contrast, the

inference phase showed comparable RAM (2.3 GB) and CPU (10.08%) usage but significantly reduced VRAM (200.0 MB) and GPU (12.5%) utilization. This marked difference underscores the feasibility of deploying the trained model in resource-aware environments, despite the higher cost associated with the training procedure.

G. Long-Term Stability Evaluation

Ascertaining the long-term stability of MTG-GAN within dynamic environments is crucial for validating its practical deployment viability. To this end, we trained the model using the initial 40% of chronologically ordered sequences and subsequently evaluated its F1-score across five consecutive 10% temporal windows spanning the remaining 50%-100% of the timeline (as illustrated in Fig. 8). The experimental results clearly indicate slow degradation in performance over time across all tested datasets (HDFS, BGL, Thunderbird, OpenStack). For instance, the F1-score on the BGL dataset moderately declined from a peak of approximately 94.5% to 90.7% in the final evaluation window, representing a total decrease of about 3.8 percentage points. Despite this decline, the degradation process is noticeably gradual, and the model maintains relatively high-performance levels during the final evaluation stages across all datasets (with F1 scores generally remaining above 89%-91%), demonstrating good resilience against significant decay.

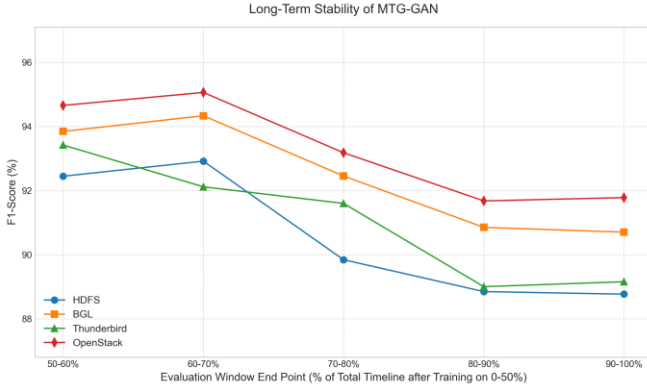


Fig. 8. MTG-GAN Performance Stability Over Time.

This noteworthy stability can be primarily attributed to the synergistic interplay of the model's key design elements. The adaptive masking mechanism enhances generalization to subsequent data variations by compelling the learning of transferable underlying patterns and suppressing overfitting to specific features from the initial training phase. The GAN framework likely facilitates the modeling of core system operational dynamics rather than superficial phenomena. Furthermore, the TCN effectively captures dependencies spanning long time windows, rendering the model less susceptible to short-term fluctuations. Concurrently, graph-based representation learning (potentially augmented by spectral loss) enables the sensitive identification of structural deviations that reflect changes in system state.

However, the gradual performance decline also underscores the inherent limitations of statically trained models when confronted with the continuous evolution of real-world

systems, particularly in adapting to concept drift and previously unseen anomaly patterns. This implies that while MTG-GAN exhibits commendable baseline stability, sustaining optimal performance long-term in highly dynamic production environments may necessitate the consideration of complementary strategies, such as periodic retraining, online learning adjustments, or the integration of explicit drift adaptation mechanisms. In summary, this study confirms that MTG-GAN possesses a solid stability foundation, while simultaneously providing a clear direction for future research focused on developing more adaptive log anomaly detection methodologies.

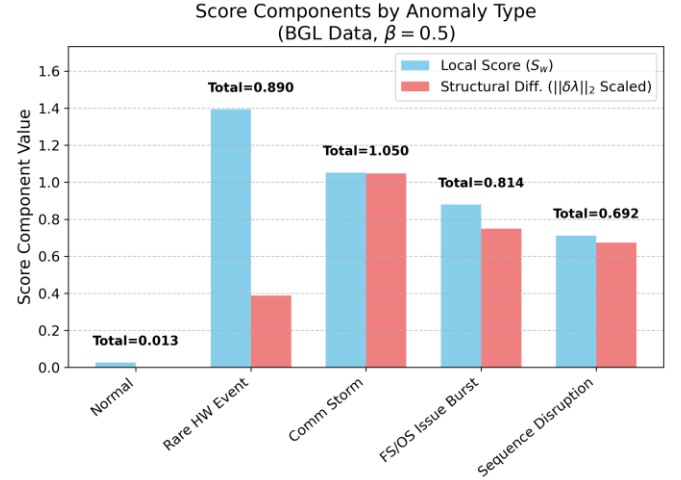


Fig. 9. Distinct Score Component Responses for Representative Anomaly Categories from BGL Data. Comparison of local (S_w) versus global (structural difference $||\delta\lambda||_2$) score components for normal behavior and representative examples of different anomaly categories identified in the BGL dataset (e.g., "Rare HW Event," "Comm Storm"). Highlights how the framework differentiates observed anomaly types based on their scoring signature.

H. Analysis of Anomaly Type Detection

To elucidate MTG-GAN's capability to distinguish between different anomaly types, we conducted a case study on typical anomaly patterns observed in the BGL dataset, focusing on the differential responses of the local pattern score (S_w) versus the global structural change score ($||\delta\lambda||_2$) (as depicted in Fig. 9).

The analysis confirmed that distinct anomaly categories indeed trigger different activation patterns across these two scoring components. For instance, sequences primarily involving rare but critical hardware events tend to significantly elevate the local pattern anomaly score (S_w) with relatively limited impact on the overall graph structure. Conversely, anomalies characterized by significant reordering of normal event sequences (sequence disruption) may result in a higher global structural difference score ($||\delta\lambda||_2$), even if the constituent events themselves are not uncommon. Events like error bursts (communication storms) typically cause deviations in both local patterns and global structure, leading to elevated scores in both components. This differential scoring response demonstrates that MTG-GAN's composite scoring mechanism

not only detects anomalies but also provides preliminary insights into their fundamental nature (whether it's a localized event anomaly or a system-level structural perturbation), thereby offering richer diagnostic information beyond a single anomaly indicator for operators.

I. Impact of Masking Rate on Model Performance

To investigate the impact of masking rate on MTG-GAN's performance, we conducted comprehensive experiments on three datasets: HDFS, BGL, and Thunderbird. The masking rate, defined as the proportion of masked adjacency matrix

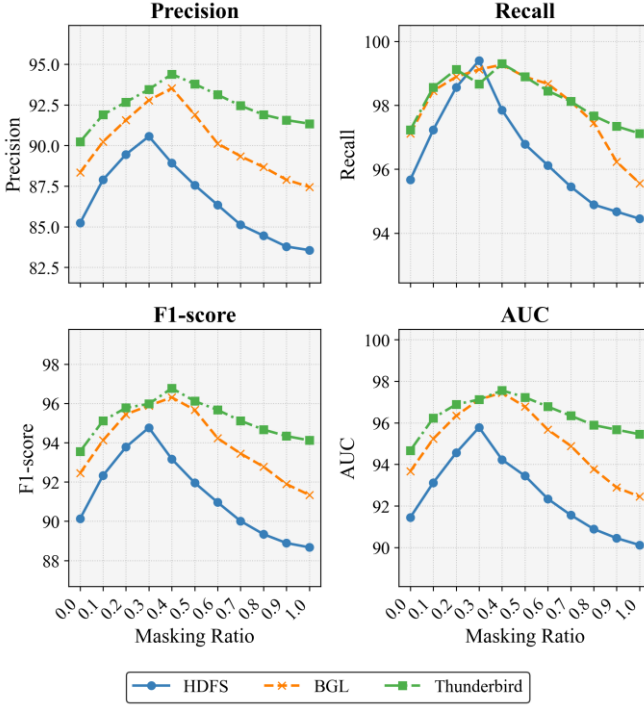


Fig. 10. Sensitivity Analysis: Effect of Masking Rate on Model Performance. Precision, Recall, F1 Score, and AUC trends on HDFS, BGL, and Thunderbird datasets as the masking rate changes from 0% to 100%. Demonstrates the existence of optimal masking ranges that balance robustness gains and information loss.

elements, was systematically varied from 0 to 1 with a step size of 0.1. We used Precision, Recall, F1 score, and Area Under Curve (AUC) as evaluation metrics to measure model performance under different masking conditions. Fig. 10 shows performance trends across different masking rates for each dataset. Experimental results demonstrate that masking rate has significant yet subtle effects on MTG-GAN's performance, with optimal rates varying by dataset and performance metric. For all three datasets, precision, recall, and F1 scores exhibit a concave trend, initially increasing then decreasing. This pattern suggests that moderate masking enhances model performance, possibly by improving robustness to noise and data loss while encouraging learning more generalizable feature representations. However, excessive masking rates lead to performance degradation, likely due to loss of critical information. On the HDFS dataset, the model achieved optimal performance at approximately 0.4

masking rate, reaching peak values of 90.56%, 99.40%, and 94.77% for precision, recall, and F1 score respectively. The BGL dataset performed best within a masking rate range of 0.5 to 0.7, while the Thunderbird dataset maintained high performance across a broader range (0.2 to 0.7). These differences may be attributed to varying dataset complexities. The Thunderbird dataset, being the most complex, enables MTG-GAN to capture intricate relationships more effectively, thus showing less sensitivity to masking rate variations. Conversely, the relatively simple HDFS dataset is more susceptible to information loss, resulting in a lower optimal masking rate. The BGL dataset, with intermediate complexity, exhibits moderate optimal masking rates. AUC curves follow trends consistent with F1 scores across all datasets, further confirming the effectiveness of the masking mechanism. Notably, optimal masking rates vary by dataset and metric, emphasizing the need to adjust masking rates for specific datasets and target metrics in practical applications to achieve excellent anomaly detection performance.

J. Weight Parameters in Masking Strategy

TABLE IV
IMPACT OF DIFFERENT MASKING STRATEGY GROWTH PATTERNS ON MODEL PERFORMANCE (BGL DATASET)

Growth Strategy	Precision (%)	Recall (%)	F1 Score (%)	AUC (%)
Linear Growth	93.52	96.27	94.87	94.35
Exponential Growth	92.45	98.68	95.46	95.48
Piecewise Growth	94.51	97.72	96.08	95.92
Periodic Growth	93.88	97.95	95.87	95.73
No Masking	90.56	94.79	92.62	91.57

To thoroughly investigate the impact of growth strategies for the weight parameter α_{adv} in the masking strategy on model performance, we conducted a series of experiments on the BGL dataset. We evaluated four different α_{adv} growth strategies: linear growth, exponential growth, piecewise growth, and periodic growth. All experiments were conducted while keeping other hyperparameters constant to ensure result reliability. Results were evaluated using precision, recall, F1 score, and AUC values, with averages taken from multiple runs to reduce randomness effects. TABLE IV shows model performance under different α_{adv} growth strategies.

Experimental results reveal several key findings. First, all variants using masking strategies significantly outperformed the no-masking baseline model, validating the effectiveness of the adaptive masking mechanism. Specifically, the no-masking model achieved an F1 score of 92.62%, while models employing masking strategies all achieved F1 scores above 94%, confirming the masking mechanism's important role in enhancing model robustness. Among different masking strategies, the piecewise growth strategy performed best, achieving an F1 score of 96.08% and AUC value of 95.92%. The advantage of this strategy might stem from its ability to adjust growth rates according to different training stages, allowing the model to fully explore feature space in early stages while focusing on refining learned patterns in later stages. Particularly in terms of precision, the piecewise growth

strategy achieved the highest level at 94.51%, indicating its effectiveness in reducing false positives and improving detection accuracy. The periodic growth strategy showed outstanding performance in recall, reaching the highest value of 97.95%. This strategy, through periodically adjusting masking weights, may enable the model to refocus on different feature patterns at various learning stages, thereby improving recognition of various anomalies. This finding has particular practical significance for applications prioritizing anomaly detection rates. The exponential growth strategy (base 1.5), while slightly lower in precision (92.45%), maintained good overall performance with high recall (98.68%). This strategy increases masking weights more rapidly in later training stages, potentially helping the model better adapt to complex anomaly patterns. In comparison, the linear growth strategy demonstrated balanced performance characteristics, achieving good balance between precision (93.52%) and recall (96.27%).

These experimental results not only validate the importance of the masking mechanism but also provide important reference for strategy selection in different application scenarios. For instance, the piecewise growth strategy might be preferable in scenarios with strict false positive requirements, while the periodic growth strategy might be more suitable for scenarios emphasizing anomaly detection. Additionally, performance differences across strategies suggest that dynamic adjustment of masking weights profoundly impacts the model's learning process, providing new research directions for further optimizing the masking mechanism. Notably, performance differences across these strategies also reflect the model's sensitivity to masking parameters. For example, the superior performance of piecewise and periodic growth strategies suggests that dynamically adjusting masking weights during training might be more beneficial for model learning than simple monotonic growth. This finding not only has direct implications for improving MTG-GAN's performance but also provides valuable reference for other mask-based models.

V. CONCLUSION

This research successfully introduces and validates MTG-GAN, a novel framework for cross-domain system log anomaly detection leveraging masked temporal graph generative adversarial networks. By innovatively integrating adaptive masking mechanisms, temporal graph structure learning, generative adversarial networks, and temporal convolutional networks, MTG-GAN effectively addresses the two core challenges in log anomaly detection: accurately capturing evolving temporal dependencies to identify emerging anomaly patterns, and ensuring robust generalization capability across diverse system environments.

The core strengths of MTG-GAN stem from the synergistic interplay of its key mechanisms. The diffused graph generation module based on generative adversarial networks enhances the model's capability to handle both known and unknown anomaly patterns. The innovative adaptive masking mechanism, by reducing dependency on specific data patterns, substantially improves its transfer generalization capability across different datasets. Furthermore, the integration of

spectral loss and feature perturbation contrastive loss preserves global topological structure, captures long-range dependencies, and provides preliminary reasoning capabilities, thereby improving result transparency and interpretability.

Extensive experimental validation across multiple real-world datasets (including system logs and industrial control system time-series data) has demonstrated MTG-GAN's superior detection performance and strong cross-domain generalization. The model particularly excels in maintaining high recall and exhibits good stability in low-data regimes. The model particularly excels in maintaining high recall and exhibits good stability in low-data regimes. Critically, the dual-component anomaly score not only enhances generalization but also differentiates anomaly types based on scoring signatures, offering system maintenance personnel preliminary diagnostic insights beyond simple anomaly labels, which holds significant practical value. Beyond these quantitative improvements, our results suggest that effective cross-domain log anomaly detection is achievable without requiring extensive system-specific retraining, which has practical implications for reducing operational overhead in organizations managing diverse IT infrastructure.

To further broaden its applicability, future work could explore optimizations for even greater scalability, particularly investigating automated or adaptive hyperparameter tuning strategies, especially for the masking mechanism, could streamline deployment across diverse datasets. Besides, integrating online learning or concept drift adaptation mechanisms presents a promising direction for ensuring sustained optimal performance in highly dynamic operational settings. Continued validation and potential adaptation across an even wider spectrum of domains, such as Internet of Things or network security logs, will further underscore its versatility.

More broadly, this work demonstrates the feasibility of building anomaly detection systems that can transfer knowledge across heterogeneous environments—a capability increasingly important as organizations adopt multi-cloud and hybrid infrastructure. The ability to deploy a single model across different systems without extensive reconfiguration addresses a key barrier to practical adoption of learning-based anomaly detection in industry settings.

REFERENCES

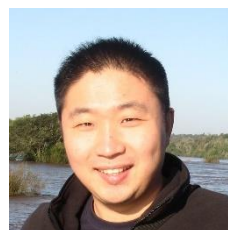
- [1] M. Du, F. Li, G. Zheng, and V. Srikumar, "DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, in CCS '17. New York, NY, USA: Association for Computing Machinery, Oct. 2017, pp. 1285–1298. doi: 10.1145/3133956.3134015.
- [2] Z. Chen, J. Liu, W. Gu, Y. Su, and M. R. Lyu, "Experience Report: Deep Learning-based System Log Analysis for Anomaly Detection," Jan. 11, 2022, arXiv: arXiv:2107.05908. doi: 10.48550/arXiv.2107.05908.
- [3] M. Platini, T. Ropars, B. Pelletier, and N. De Palma, "LogFlow: Simplified Log Analysis for Large Scale Systems," in *Proceedings of the 22nd International Conference on Distributed Computing and Networking*, in ICDCN '21. New York, NY, USA: Association for Computing Machinery, Jan. 2021, pp. 116–125. doi: 10.1145/3427796.3427808.
- [4] Y. Chen, N. Luktaran, and D. Lv, "LogLS: Research on System Log Anomaly Detection Method Based on Dual LSTM," *Symmetry*, vol. 14, no. 3, Art. no. 3, Mar. 2022, doi: 10.3390/sym14030454.

- [5] S. He, J. Zhu, P. He, and M. R. Lyu, "Experience Report: System Log Analysis for Anomaly Detection," in *2016 IEEE 27th International Symposium on Software Reliability Engineering (ISSRE)*, Oct. 2016, pp. 207–218. doi: 10.1109/ISSRE.2016.21.
- [6] H. Yang, D. Sun, Y. Wang, and W. Huang, "DSGN: Log-based anomaly diagnosis with dynamic semantic gate networks," *Information Sciences*, vol. 680, p. 121174, Oct. 2024, doi: 10.1016/j.ins.2024.121174.
- [7] M. Landauer, S. Onder, F. Skopik, and M. Wurzenberger, "Deep Learning for Anomaly Detection in Log Data: A Survey," *Machine Learning with Applications*, vol. 12, p. 100470, Jun. 2023, doi: 10.1016/j.mlwa.2023.100470.
- [8] X. Ma et al., "A Comprehensive Survey on Graph Anomaly Detection With Deep Learning," *IEEE Transactions on Knowledge and Data Engineering*, vol. 35, no. 12, pp. 12012–12038, Dec. 2023, doi: 10.1109/TKDE.2021.3118815.
- [9] X. Xie, S. Jian, C. Huang, F. Yu, and Y. Deng, "LogRep: Log-based Anomaly Detection by Representing both Semantic and Numeric Information in Raw Messages," in *2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE)*, Oct. 2023, pp. 194–206. doi: 10.1109/ISSRE59848.2023.00015.
- [10] V.-H. Le and H. Zhang, "Log-based anomaly detection with deep learning: how far are we?," in *Proceedings of the 44th International Conference on Software Engineering*, in ICSE '22. New York, NY, USA: Association for Computing Machinery, Jul. 2022, pp. 1356–1367. doi: 10.1145/3510003.3510155.
- [11] X. Li, P. Chen, L. Jing, Z. He, and G. Yu, "SwissLog: Robust Anomaly Detection and Localization for Interleaved Unstructured Logs," *IEEE Transactions on Dependable and Secure Computing*, vol. 20, no. 4, pp. 2762–2780, Jul. 2023, doi: 10.1109/TDSC.2022.3162857.
- [12] M. Landauer, F. Skopik, and M. Wurzenberger, "A Critical Review of Common Log Data Sets Used for Evaluation of Sequence-Based Anomaly Detection Techniques," *Proc. ACM Softw. Eng.*, vol. 1, no. FSE, p. 1354–1375, Jul. 2024, doi: 10.1145/3660768.
- [13] X. Xie, Z. Jin, J. Wang, L. Yang, Y. Lu, and T. Li, "Confidence guided anomaly detection model for anti-concept drift in dynamic logs," *Journal of Network and Computer Applications*, vol. 162, p. 102659, Jul. 2020, doi: 10.1016/j.jnca.2020.102659.
- [14] L. Bao, Q. Li, P. Lu, J. Lu, T. Ruan, and K. Zhang, "Execution anomaly detection in large-scale systems through console log analysis," *Journal of Systems and Software*, vol. 143, pp. 172–186, Sep. 2018, doi: 10.1016/j.jss.2018.05.016.
- [15] P. Jia, S. Cai, B. C. Ooi, P. Wang, and Y. Xiong, "Robust and Transferable Log-based Anomaly Detection," *Proc. ACM Manag. Data*, vol. 1, no. 1, p. 1–26, May 2023, doi: 10.1145/3588918.
- [16] M. Landauer, M. Wurzenberger, F. Skopik, G. Settanni, and P. Filzmoser, "Dynamic log file analysis: An unsupervised cluster evolution approach for anomaly detection," *Computers & Security*, vol. 79, pp. 94–116, Nov. 2018, doi: 10.1016/j.cose.2018.08.009.
- [17] A. A. Cook, G. Misirlı, and Z. Fan, "Anomaly Detection for IoT Time-Series Data: A Survey," *IEEE Internet of Things Journal*, vol. 7, no. 7, pp. 6481–6494, Jul. 2020, doi: 10.1109/IIOT.2019.2958185.
- [18] R. Vinayakumar, K. P. Soman, and P. Poornachandran, "Long short-term memory based operation log anomaly detection," in *2017 International Conference on Advances in Computing, Communications and Informatics (ICACCI)*, Sep. 2017, pp. 236–242. doi: 10.1109/ICACCI.2017.8125846.
- [19] R. Larisch, J. Vitay, and F. H. Hamker, "Detecting Anomalies in System Logs With a Compact Convolutional Transformer," *IEEE Access*, vol. 11, pp. 113464–113479, 2023, doi: 10.1109/ACCESS.2023.3323252.
- [20] H. Li and Y. Li, "LogSpy: System Log Anomaly Detection for Distributed Systems," in *2020 International Conference on Artificial Intelligence and Computer Engineering (ICAICE)*, Oct. 2020, pp. 347–352. doi: 10.1109/ICAICE51518.2020.00073.
- [21] H. Guo, S. Yuan, and X. Wu, "LogBERT: Log Anomaly Detection via BERT," in *2021 International Joint Conference on Neural Networks (IJCNN)*, Jul. 2021, pp. 1–8. doi: 10.1109/IJCNN52387.2021.9534113.
- [22] S. Han and S. S. Woo, "Learning Sparse Latent Graph Representations for Anomaly Detection in Multivariate Time Series," in *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, in KDD '22. New York, NY, USA: Association for Computing Machinery, Aug. 2022, pp. 2977–2986. doi: 10.1145/3534678.3539117.
- [23] W. Xu, L. Huang, A. Fox, D. Patterson, and M. I. Jordan, "Detecting large-scale system problems by mining console logs," in *Proceedings of the ACM SIGOPS 22nd symposium on Operating systems principles*, in SOSP '09. New York, NY, USA: Association for Computing Machinery, Oct. 2009, pp. 117–132. doi: 10.1145/1629575.1629587.
- [24] R. Vaarandi and M. Pihelgas, "LogCluster - A data clustering and pattern mining algorithm for event logs," in *2015 11th International Conference on Network and Service Management (CNSM)*, Nov. 2015, pp. 1–7. doi: 10.1109/CNSM.2015.7367331.
- [25] W. Meng et al., "LogAnomaly: Unsupervised Detection of Sequential and Quantitative Anomalies in Unstructured Logs," in *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence, IJCAI 2019, Macao, China, August 10-16, 2019*, 2019, pp. 4739–4745. doi: 10.24963/IJCAI.2019/658.
- [26] X. Zhang et al., "Robust log-based anomaly detection on unstable log data," in *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, in ESEC/FSE 2019. New York, NY, USA: Association for Computing Machinery, Aug. 2019, pp. 807–817. doi: 10.1145/3338906.3338931.
- [27] L. Yan, C. Luo, and R. Shao, "Discrete log anomaly detection: A novel time-aware graph-based link prediction approach," *Information Sciences*, vol. 647, p. 119576, Nov. 2023, doi: 10.1016/j.ins.2023.119576.
- [28] P. Veličković, W. Fedus, W. L. Hamilton, P. Liò, Y. Bengio, and R. D. Hjelm, "Deep Graph Infomax," in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2019. [Online]. Available: <https://openreview.net/forum?id=rklz9iAcKQ>
- [29] Z. Hou et al., "GraphMAE: Self-Supervised Masked Graph Autoencoders," in *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, in KDD '22. New York, NY, USA: Association for Computing Machinery, Aug. 2022, pp. 594–604. doi: 10.1145/3534678.3539321.
- [30] C. Almodovar, F. Sabrina, S. Karimi, and S. Azad, "LogFiT: Log Anomaly Detection Using Fine-Tuned Language Models," *IEEE Transactions on Network and Service Management*, vol. 21, no. 2, pp. 1715–1723, Apr. 2024, doi: 10.1109/TNSM.2024.3358730.
- [31] J. Qi et al., "LogEncoder: Log-Based Contrastive Representation Learning for Anomaly Detection," *IEEE Transactions on Network and Service Management*, vol. 20, no. 2, pp. 1378–1391, Jun. 2023, doi: 10.1109/TNSM.2023.3239522.
- [32] X. Han and S. Yuan, "Unsupervised Cross-system Log Anomaly Detection via Domain Adaptation," in *Proceedings of the 30th ACM International Conference on Information & Knowledge Management*, in CIKM '21. New York, NY, USA: Association for Computing Machinery, Oct. 2021, pp. 3068–3072. doi: 10.1145/3459637.3482209.
- [33] R. Chen et al., "LogTransfer: Cross-System Log Anomaly Detection for Software Systems with Transfer Learning," in *2020 IEEE 31st International Symposium on Software Reliability Engineering (ISSRE)*, Oct. 2020, pp. 37–47. doi: 10.1109/ISSRE5003.2020.00013.
- [34] C. Zhang, T. Jia, G. Shen, P. Zhu, and Y. Li, "MetaLog: Generalizable Cross-System Anomaly Detection from Logs with Meta-Learning," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, in ICSE '24. New York, NY, USA: Association for Computing Machinery, Apr. 2024, pp. 1–12. doi: 10.1145/3597503.3639205.
- [35] K. Gong, S. Luo, L. Pan, L. Zhang, Y. Zhang, and H. Yu, "LogETA: Time-aware cross-system log-based anomaly detection with inter-class boundary optimization," *Future Generation Computer Systems*, vol. 157, pp. 16–28, Aug. 2024, doi: 10.1016/j.future.2024.03.028.
- [36] P. Wang, X. Zhang, Z. Cao, W. Xu, and W. Li, "LogGT: Cross-system log anomaly detection via heterogeneous graph feature and transfer learning," *Expert Systems with Applications*, vol. 251, p. 124082, Oct. 2024, doi: 10.1016/j.eswa.2024.124082.
- [37] J. Zhu, S. He, P. He, J. Liu, and M. R. Lyu, "Loghub: A Large Collection of System Log Datasets for AI-driven Log Analytics," in *2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE)*, Oct. 2023, pp. 355–366. doi: 10.1109/ISSRE59848.2023.00071.
- [38] A. P. Mathur and N. O. Tippenhauer, "SWaT: a water treatment testbed for research and training on ICS security," in *2016 International Workshop on Cyber-physical Systems for Smart Water Networks (CySWater)*, Apr. 2016, pp. 31–36. doi: 10.1109/CySWater.2016.7469060.

- [39] S. Li, W. Meng, S. He, J. Bi, and G. Liu, "STAGED: A Spatial-Temporal Aware Graph Encoder-Decoder for Fault Diagnosis in Industrial Processes," *IEEE Transactions on Industrial Informatics*, vol. 20, no. 2, pp. 1742–1752, Feb. 2024, doi: 10.1109/TII.2023.3281083.
- [40] C. M. Ahmed, V. R. Palleti, and A. P. Mathur, "WADI: a water distribution testbed for research in the design of secure cyber physical systems," in *Proceedings of the 3rd International Workshop on Cyber-Physical Systems for Smart Water Networks*, in CySWATER '17. New York, NY, USA: Association for Computing Machinery, Apr. 2017, pp. 25–28. doi: 10.1145/3055366.3055375.
- [41] L. Liao, K. Zhu, J. Luo, and J. Cai, "LogBASA: Log Anomaly Detection Based on System Behavior Analysis and Global Semantic Awareness," *International Journal of Intelligent Systems*, vol. 2023, no. 1, p. 3777826, 2023, doi: 10.1155/2023/3777826.
- [42] J. Qi *et al.*, "LogGPT: Exploring ChatGPT for Log-Based Anomaly Detection," in *2023 IEEE International Conference on High Performance Computing & Communications, Data Science & Systems, Smart City & Dependability in Sensor, Cloud & Big Data Systems & Application (HPCC/DSS/SmartCity/DependSys)*, Dec. 2023, pp. 273–280. doi: 10.1109/HPCC-DSS-SmartCity-DependSys60770.2023.00045.
- [43] A. Farzad and T. A. Gulliver, "Unsupervised log message anomaly detection," *ICT Express*, vol. 6, no. 3, pp. 229–237, Sep. 2020, doi: 10.1016/j.icte.2020.06.003.
- [44] Z. Zhao, W. Niu, X. Zhang, R. Zhang, Z. Yu, and C. Huang, "Trine: Syslog anomaly detection with three transformer encoders in one generative adversarial network," *Applied Intelligence*, vol. 52, no. 8, pp. 8810–8819, Jun. 2022, doi: 10.1007/s10489-021-02863-9.
- [45] P. S. Kalaki, A. Shameli-Sendi, and B. K. E. Abbasi, "Anomaly detection on OpenStack logs based on an improved robust principal component analysis model and its projection onto column space," *Software: Practice and Experience*, vol. 53, no. 3, pp. 665–681, 2023, doi: 10.1002/spe.3164.
- [46] Z. Zhang, Z. Geng, and Y. Han, "Graph Structure Change-Based Anomaly Detection in Multivariate Time Series of Industrial Processes," *IEEE Transactions on Industrial Informatics*, vol. 20, no. 4, pp. 6457–6466, Apr. 2024, doi: 10.1109/TII.2023.3347000.
- [47] H. Kim, S. Kim, S. Min, and B. Lee, "Contrastive Time-Series Anomaly Detection," *IEEE Transactions on Knowledge and Data Engineering*, vol. 36, no. 10, pp. 5053–5065, Oct. 2024, doi: 10.1109/TKDE.2023.3335317.
- [48] J. Cai, Z. Wei, and J. Luo, "ICS Anomaly Detection Based on Sensor Patterns and Actuator Rules in Spatiotemporal Dependency," *IEEE Transactions on Industrial Informatics*, vol. 20, no. 8, pp. 10647–10656, Aug. 2024, doi: 10.1109/TII.2024.3393528.
- [49] J. Li, Z. Yu, Q. Jiang, and Z. Cao, "One-Class Classification Constraint in Reconstruction Networks for Multivariate Time Series Anomaly Detection," *IEEE Transactions on Instrumentation and Measurement*, vol. 74, pp. 1–13, 2025, doi: 10.1109/TIM.2025.3548251.
- [50] M. Yao, D. Tao, P. Qi, and R. Gao, "Rethinking Discrepancy Analysis: Anomaly Detection via Meta-Learning Powered Dual-Source Representation Differentiation," *IEEE Transactions on Automation Science and Engineering*, vol. 22, pp. 8579–8592, 2025, doi: 10.1109/TASE.2024.3486688.
- [51] R. Li, Z. Liu, X. Zhu, L. Li, and X. Cao, "Detecting Multivariate Time Series Anomalies With Cascade Decomposition Consistency," *IEEE Transactions on Instrumentation and Measurement*, vol. 74, pp. 1–14, 2025, doi: 10.1109/TIM.2025.3547479.
- [52] M. Li *et al.*, "SA2E-AD: A Stacked Attention Autoencoder for Anomaly Detection in Multivariate Time Series," *ACM Trans. Knowl. Discov. Data*, vol. 18, no. 7, p. 174:1–174:15, Jun. 2024, doi: 10.1145/3653677.
- [53] Y. Li *et al.*, "Accurate Anomaly Detection Leveraging Knowledge-enhanced GAT," in *2024 IEEE International Conference on Web Services (ICWS)*, Jul. 2024, pp. 568–577. doi: 10.1109/ICWS62655.2024.00077.
- [54] F. Peng *et al.*, "MGRD: Multigranularity Reconstruction Deviation Modeling for Time Series Anomaly Detection," *IEEE Transactions on Instrumentation and Measurement*, vol. 74, pp. 1–11, 2025, doi: 10.1109/TIM.2025.3545174.



Jing Zhang received the B.S. degree from Shanghai Institute of Technology, China, in 2023. He is currently pursuing the M.E. degree with the School of Information Science and Engineering, Shandong Normal University. His research interests include anomaly detection and multivariate time series analysis.



Chao Luo received the PhD degree in computer science from Dalian University of Technology, China, 2013. He is currently a professor in the School of Information Science and Engineering, Shandong Normal University, China. He has published more than 60 refereed papers. His research interests include machine learning, complex system and time series analysis.



Rui Shao graduated from Shandong University with a Master's degree in Computer Software and Theory. He is currently employed by China Mobile Communications Group Shandong Co., Ltd., mainly engaged in wireless network optimization and planning work for 5G/4G networks. At present, the main research directions are 5G communication network planning, strategy optimization, SON, anomaly recognition, and network structure optimization.